

Temporal Difference Learning 1

Sarsa, Q-learning, and the on/off-policy distinction

Seung Ho Go

August 16, 2026

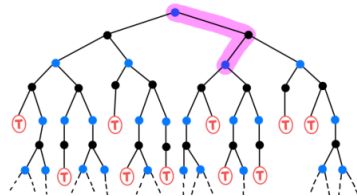
Table of contents

- 1 Where TD sits
- 2 Three targets: MC, Sarsa, Q-learning
- 3 The two algorithms
- 4 On-policy vs. off-policy

Recap: TD fills the empty cell

Three axes separate every method we have seen: **model** · **backup width** · **bootstrapping**.

	model	backup	bootstrap
DP	needed	full	yes
MC	free	sample, to the end	no
TD	free	sample, 1 step	yes



TD backup: one sampled step, then bootstrap.

TD is not a new invention — it is MC's *sampling* combined with DP's *bootstrapping*.

Why that combination is worth having

Bootstrapping — from DP

Update from an **estimate** of the next state, so we **never wait for the final outcome**.

⇒ works **online**, and on tasks that never terminate.

Sampling — from MC

Learn from **experienced transitions**, not from the full model.

⇒ **model-free**: no transition probabilities required.

*MC must reach a terminal state before it can compute G_t . TD only needs **one transition**: $S_t \rightarrow S_{t+1}$.*

TD is still **tabular** and **model-free**, and TD Policy Iteration is still **GPI** — only now one step of experience is enough to run a round of it.

The three policy evaluations differ in *one place*

All three share the shape $Q \leftarrow Q + \alpha[\text{target} - Q]$. Only the **target** changes.

Monte Carlo — On-Policy MC Prediction

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)]$$

Sarsa — On-Policy TD Prediction

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Q-learning — Off-Policy TD Prediction (Watkins 1989)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

G_t (whole episode) $\rightarrow Q(S', A')$ (the action we *will take*) $\rightarrow \max_a Q(S', a)$ (the *best* action)

Same shape, different Bellman equation

Sarsa learns Q^π

Target uses A_{t+1} , the action the **current policy** actually picks.

- ⇒ Bellman **expectation** equation
- ⇒ **on-policy**

Q-learning learns Q^*

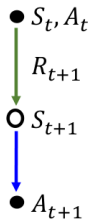
Target uses $\max_a$, the **best** action regardless of what we will do.

- ⇒ Bellman **optimality** equation
- ⇒ **off-policy**

Q-learning aims **directly at** Q^* instead of at the current policy's Q^π — which is why it improves faster, and why it became **the** canonical RL algorithm.

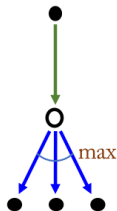
Backup diagrams: what each target looks at

Sarsa



follows **one** chosen action

Q-learning



looks at **all** actions, takes the max

Both stop after **one step** — that is what makes them TD.

The difference is the **last hop**:

- Sarsa walks to A_{t+1} and uses *that* value
- Q-learning fans out over every action at S_{t+1} and keeps the *largest*

one edge vs. a fan — that single change is the whole difference.

What sample does each one need?

Sarsa — five items

$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

State, Action, Reward, State, Action

⇒ that spelling *is* the name **SARSA**

Q-learning — four items

$(S_t, A_t, R_{t+1}, S_{t+1})$

the next action is **not needed**:

the target takes $\max_a$ instead

Sarsa must commit to A_{t+1} before it can update; Q-learning never has to.

Sarsa

Sarsa (on-policy TD control)

Initialize $Q(S, A)$ arbitrarily, $Q(\text{terminal}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g. ϵ -greedy)

Repeat (for each step of episode):

Take action A ; observe R, S'

Choose A' from S' using policy derived from Q (e.g. ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A'$

Until S is terminal

The action used in the target is the same one we are **about to take** — behavior policy = target policy.

Q-learning

Q-learning (off-policy TD control)

Initialize $Q(S, A)$ arbitrarily, $Q(\text{terminal}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Repeat (for each step of episode):

Choose A from S using policy derived from Q (e.g. ϵ -greedy)

Take action A ; observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

Until S is terminal

The action in the target is chosen by **max**, not by the behavior policy — so the two policies come apart.

Side by side: only three lines differ

	Sarsa	Q-learning
Where A is chosen	<i>before</i> the step loop	<i>inside</i> the step loop
Next action A'	chosen by ϵ -greedy, then used	not needed in the update
Target	$R + \gamma Q(S', A')$	$R + \gamma \max_a Q(S', a)$
Carry-over	$S \leftarrow S', \quad A \leftarrow A'$	$S \leftarrow S'$ only
Bellman equation	expectation	optimality
Learns	Q^π	Q^*
Policy type	on-policy	off-policy

Every row is a consequence of the one change in the target.

Two policies hiding in one algorithm

Target policy

The policy the agent is **trying to learn** — the one whose value function we are estimating. It lives **inside the target**.

Behavior policy

The policy actually used to **choose actions and generate the data** — the episodes we learn from.

On-policy: target = behavior

Sarsa

Off-policy: target $\pi \neq$ behavior μ

Q-learning

Reading it off the two targets

Sarsa — the target's A' comes from S' under the **behavior policy** ϵ -greedy(Q):

$$R + \gamma Q(S', A') \quad A' \sim \epsilon\text{-greedy}(Q)$$

Q-learning — the target's action is whichever **maximizes** Q , chosen by no behavior policy at all:

$$R + \gamma \max_a Q(S', a) \quad a = \arg \max_a Q(S', a)$$

In Q-learning the target policy is **greedy** (deterministic) while the behavior policy stays **ϵ -greedy** (stochastic).

Note: after updating, the next action is still drawn from the behavior policy. The split happens **only inside the target**.

Why off-policy is powerful

1. It can reuse old experience

$\max_a$ ranges over **every** action at S' , most of whose values were learned **long ago**. On-policy throws its data away: once the policy improves, past experience no longer matches the policy being evaluated.

2. It explores while still learning the optimum

The target follows a policy **other than** the one generating data, so the agent can keep exploring with ϵ -greedy and still learn Q^* .

On-policy finds the best action *within the current policy* — which can be only **locally** optimal. Off-policy aims at the **global** optimum.

Takeaways

- ① **TD** = MC's **sampling** + DP's **bootstrapping**: learn from a **single transition**, no model, no waiting for the episode to end.
- ② All three methods share $Q \leftarrow Q + \alpha[\text{target} - Q]$; only the **target** differs: G_t / $Q(S', A')$ / $\max_a Q(S', a)$.
- ③ **Sarsa** uses the Bellman **expectation** equation and learns Q^π ; **Q-learning** uses the Bellman **optimality** equation and learns Q^* .
- ④ Sarsa needs the 5-tuple **(S,A,R,S,A)** — hence its name; Q-learning needs only four.
- ⑤ **On-policy** (target = behavior) vs. **off-policy** (target $\neq$ behavior): off-policy reuses past data and explores more freely.

Next: how the two actually behave — the **cliff walking** comparison, and bias vs. variance of MC and TD.

Thank you

Questions?